MetaVibing

A Field Manual for Evolving Your AI Collaborator

Mirco A. Mannucci, PhD, HoloMathics, LLC

MetaVibing v0.1.0-alpha.1 — Alpha Research Preview

The Idea, in One Page

Your AI collaborator makes a mistake. You correct it. Next week, in a new session, the correction is gone — it lived in your prompt, and prompts don't survive the conversation that produced them.

Prompt engineering improves the current conversation. MetaVibing improves the next one.

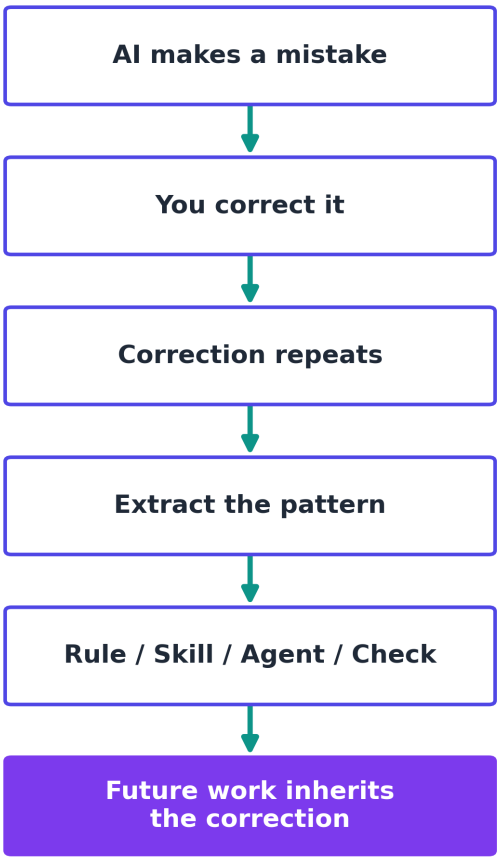
The method: when a correction proves valuable enough that you'd give it a second or third time, stop repeating it and give it a durable, checked-into-the-repository form instead —

- a fact `CLAUDE.md` should hold,
- a constraint scoped to the files where it actually applies,
- a procedure worth performing the same way every time,
- a role worth delegating to a specialist who didn't write the code,
- an invariant worth checking mechanically instead of asking a model to remember it.

That's the whole idea. Everything from here is *how* — in enough concrete detail to actually do it, and an honest account of what's proven so far and what isn't.

The Core Loop

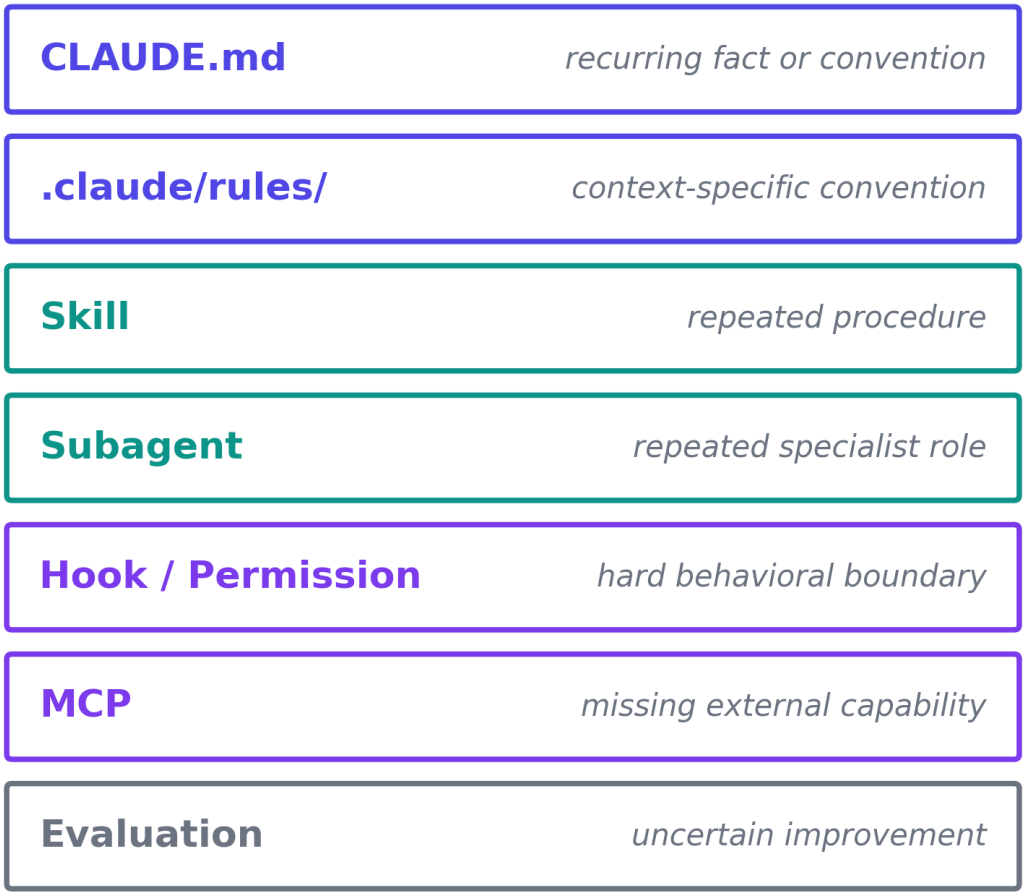
```
AI makes a mistake
       ↓
You correct it
       ↓
Correction repeats
       ↓
Extract the pattern
       ↓
Rule / Skill / Agent / Check
       ↓
Future work inherits the correction
```



The Meta-Stack

Different kinds of friction call for different kinds of artifact:

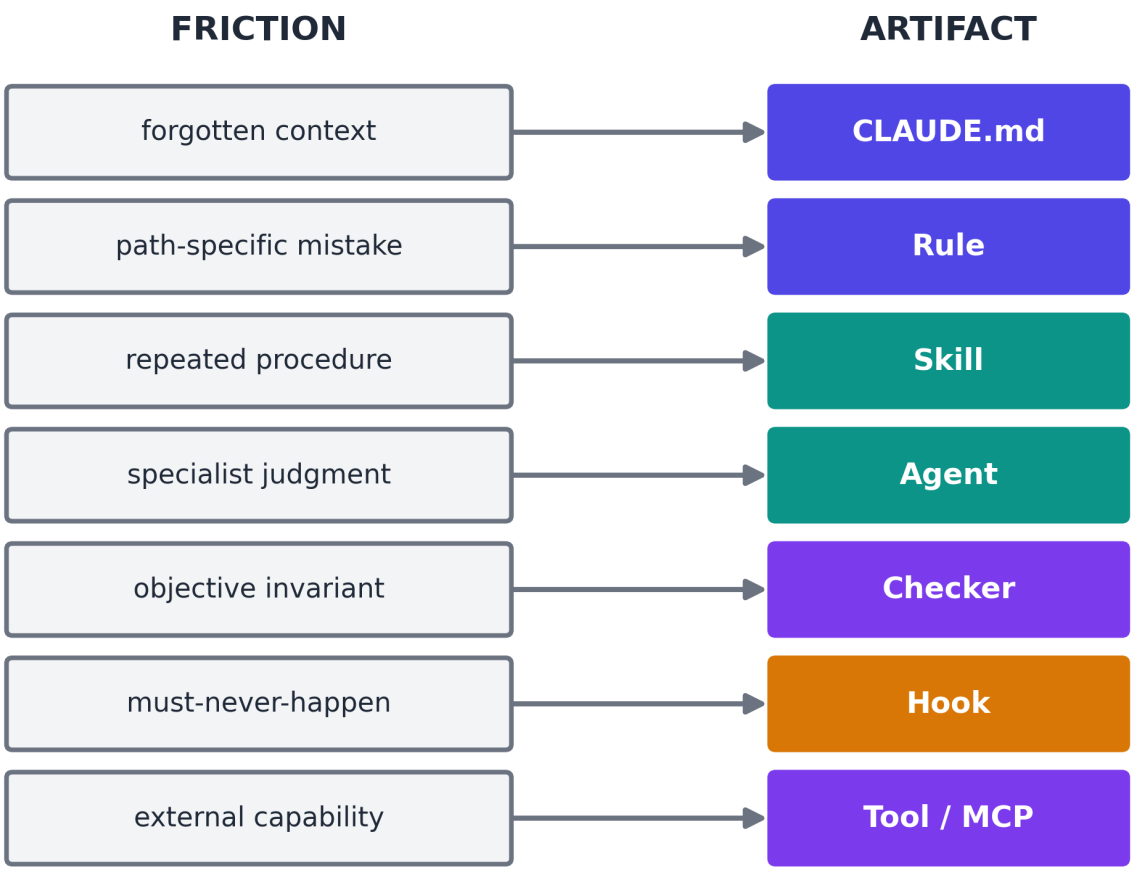
```
Recurring fact or convention    →  CLAUDE.md
Context-specific convention     →  .claude/rules/
Repeated procedure              →  Skill
Repeated specialist role        →  Subagent
Hard behavioral boundary        →  Permission / Hook
Missing external capability     →  MCP
Uncertain improvement           →  Evaluation
```



The Friction → Artifact Map

The same mapping, read the other way — starting from what went wrong:

```
FRICTION
   ■
   ■■■ forgotten context      →  CLAUDE.md
   ■■■ path-specific mistake  →  Rule
   ■■■ repeated procedure     →  Skill
   ■■■ specialist judgment    →  Agent
   ■■■ objective invariant    →  Checker
   ■■■ must-never-happen      →  Hook
   ■■■ external capability    →  Tool / MCP
```



A Five-Minute Example

TaskFlow — the companion repository's sandbox app — has a route handler that talks to the database directly:

```
@app.post("/tasks/", response_model=Task, status_code=201)
def create_task(task: Task, session: Session = Depends(get_session)):
    session.add(task)
    session.commit()
    session.refresh(task)
    return task
```

It works. It's also a pattern a growing project wouldn't want repeated: harder to test in isolation, harder to change persistence later. The ordinary fix is to ask Claude for a new feature, notice it repeats the same pattern, and say: *"Don't put database access in the route handler — keep that in the repository layer."* That correction normally dies with the conversation.

PRACTICE

Instead, look at what's already checked into this repository: `.claude/rules/taskflow.md`, scoped to `examples/taskflow/**/*`, states exactly that constraint. Because it's a native Claude Code Rule — not a comment, not a wiki page — it loads automatically the moment Claude works with a matching file. Ask for a new feature now (`/ship-change <task>`), and the Rule shapes the work without you restating anything.

One rung further: once a constraint can be checked mechanically, stop asking a model whether it was obeyed. `python tools/architecture-checker/checker.py examples/taskflow` walks the actual code and reports every violation of this exact rule — judgment converted into infrastructure.

This is the entire method, once. The full hands-on version — including the failure case, and what to do when the Rule *doesn't* hold — is `docs/10-minute-metavibe.md` in the companion repository.

Status & Evidence

MetaVibing v0.1 is an alpha research preview. The methodology, the Claude Code configuration (`.claude/rules/`, Skills, and the `final-reviewer` Agent), the TaskFlow specimen, the deterministic architecture checker, and the evaluation protocol are implemented. The controlled A/B pilot has been designed — frozen tasks, held-out acceptance tests, a grading rubric, a machine-readable protocol — but not yet executed. **No empirical performance-uplift claim is made in this release.** Readers should understand the following before proceeding.

Core Claim

> ***A practitioner who applies the MetaVibing discipline — CLAUDE.md, path-scoped Rules, Skills, a specialist Agent, and a deterministic checker — will produce a higher rate of first-try successes on architecturally constrained tasks, commit fewer repeated architectural violations per session, and require fewer human correction turns per feature request, compared to the same practitioner operating without any MetaVibing artifacts.***

This is a quantitative, falsifiable claim. The evaluation infrastructure described in `evals/baseline/README.md` exists to test it against a controlled sandbox. If measured uplift is weak, the claim must be revised. The eval does not exist to confirm the claim; it exists to challenge it.

Target User

Primary user: Technical creators, researchers, and small lab builders using AI tools for multi-step intellectual or software artifacts.

For the baseline evaluation, this is operationalised as: a practitioner with sufficient engineering fluency to run `pytest`, read FastAPI code, and follow a CLAUDE.md prompt. They do not need to

know how MetaVibing works at the start of the evaluation — the protocol hands them the artifact stack or withholds it depending on the condition.

Secondary audience: Engineering managers evaluating team-scale AI coding discipline; AI/agent researchers interested in self-modifying operational contexts.

Baseline Evidence

EVIDENCE

TaskFlow baseline: 8/8 tests passing on the alpha release commit. This confirms the companion repository's declared test suite runs cleanly. It does not show anything about the pilot's release gates, the comparison between Condition A and Condition B, or the 3×3×2 baseline protocol — none of those runs have been completed.

What Has Not Been Validated

Readers must understand what this alpha does **not** claim:

The 3×3×2 baseline protocol has not been run. The evaluation charter specifies 3 tasks × 3 trials × 2 conditions = 18 task-trials. No Condition A or Condition B trials have been executed. No M1 (architectural violation rate), M2 (correction turns), or M3 (pytest pass rate on first submission) data has been collected for any trial.

This manual has not been exhaustively copyedited. It is a working draft, revised repeatedly against real evidence rather than polished once and left alone.

Preface: Vibe Coding Was Only the Beginning

Vibe coding gave us a remarkable new interaction with software.

Instead of translating every intention manually into syntax, we could increasingly say:

> *Build this.*
>
> *Fix this.*
>
> *Try another architecture.*
>
> *Make the interface less ugly.*
>
> *Find the bug.*

And the machine would act.

That was a genuine shift.

But it still preserved an old assumption:

the programmer improves the programmer.

Claude writes the application, while the human remains responsible for improving Claude's working methods.

That assumption is now becoming obsolete.

Claude Code can be given persistent project memory, reusable Skills, specialist subagents, deterministic hooks, external MCP capabilities, plugins, parallel workers, and evaluation machinery. Claude can also create and modify most of these artifacts itself.

So a second loop becomes possible:

```
                    PRODUCT LOOP

Human intention
      ↓
    Claude
      ↓
    Code
      ↓
 Tests / results
      ↓
 Better product

                  META LOOP

Experience with Claude
      ↓
Observe friction
      ↓
Claude analyzes its own environment
      ↓
Rules / Skills / Agents / Hooks / Tools
      ↓
Better Claude behavior
      ↓
Future Claude sessions
```

I call working deliberately in this second loop:

MetaVibing

Vibe coding means collaborating with an AI to create the artifact.

MetaVibing means collaborating with the AI to improve the intelligence system that creates the artifact.

And once this becomes systematic, we enter something larger:

The MetaAgents Era

Agents no longer merely perform work.

They increasingly participate in the design, extension, evaluation, specialization, and governance of the agentic systems in which they themselves operate.

This manual explains how to do that practically with Claude Code.

Part I — What MetaVibing Actually Is

1. Meta-code versus product code

Imagine that Claude repeatedly forgets to run an integration test before declaring a feature finished.

You could write:

> *Remember to run the integration tests.*

That is an ordinary prompt.

You could add:

```
Before claiming a feature is complete, run the relevant integration tests.
```

to CLAUDE.md.

That is meta-code.

Now imagine that Claude repeatedly performs the same release sequence:

- inspect changes;
- run tests;
- bump version;
- update changelog;
- build artifacts;
- inspect git diff;
- create commit.

You could explain those seven steps every time.

Or you could create:

```
.claude/skills/release/SKILL.md
```

and thereafter invoke:

```
/release
```

That is meta-code.

Suppose Claude has a habit of touching generated migrations even though your project forbids it.

You could remind it.

Or you could install a PreToolUse hook that rejects edits to the migration directory before they happen.

That is stronger meta-code.

Hooks exist precisely because some behavior should be deterministic rather than left to model discretion.

The object being programmed has changed.

In ordinary coding:

```
CODE → APPLICATION BEHAVIOR
```

In MetaVibing:

```
META-CODE → AGENT BEHAVIOR → APPLICATION BEHAVIOR
```

2. The fundamental MetaVibing transformation

Whenever Claude does something badly, ask:

> *What artifact would make this correction unnecessary next time?*

That one question contains most of the method.

A useful transformation table is:

```
Recurring fact or convention
        ↓
CLAUDE.md

Context-specific convention
        ↓
.claude/rules/

Repeated procedure
        ↓
Skill

Repeated specialist role
        ↓
Subagent

Hard behavioral boundary
        ↓
Permission / Hook

Missing external capability
        ↓
MCP

Reusable bundle of capabilities
        ↓
Plugin

Complex parallel reasoning
        ↓
Subagents / Agent Team

Uncertain improvement
        ↓
Evaluation
```

```
Repeated meta-maintenance
        ↓
MetaAgent
```

This is the Failure → Artifact principle.

3. The Three-Strikes Rule

A practical rule:

First occurrence — Correct it

Claude does something undesirable.

Tell Claude what went wrong.

Second occurrence — Diagnose it

Ask whether the failure is accidental or structural.

Was Claude missing:

- knowledge?
- context?
- a workflow?
- a specialist role?
- a capability?
- a hard constraint?
- an evaluation criterion?

Third occurrence — Externalize it

There should now be an artifact.

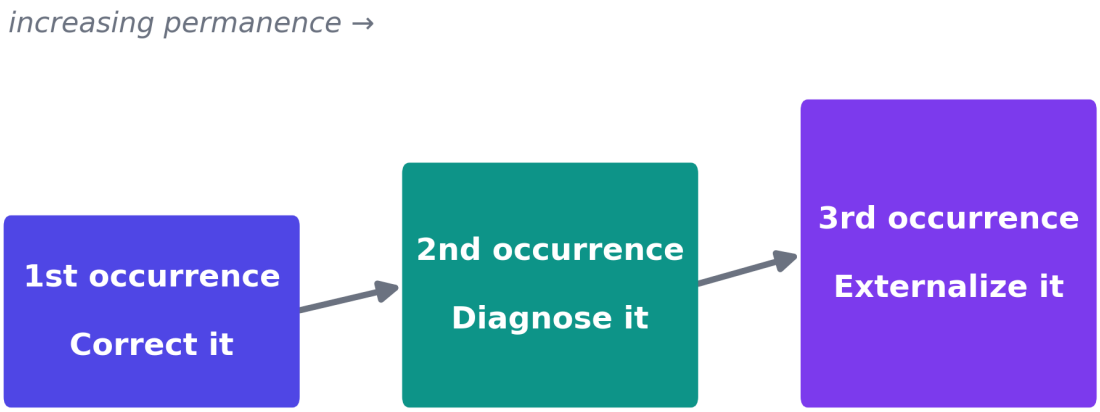
Do not suffer the same class of failure indefinitely through conversation.

Create something persistent.

My preferred formulation is:

PRINCIPLE

> *Never suffer the same Claude failure three times.*



A mature MetaVibing environment gradually converts human irritation into machine-readable institutional memory.

Part II — The Claude Meta-Stack

Claude Code gives us several qualitatively different mechanisms.

The mistake is to treat all of them as "prompts."

They are not.

4. Layer One — CLAUDE.md: Persistent Doctrine

Every Claude Code session begins with a fresh working context, but Claude Code can carry project knowledge between sessions using CLAUDE.md and auto-memory. Project instructions can live in ./CLAUDE.md or ./.claude/CLAUDE.md, while .claude/rules/ can modularize or path-scope instructions.

Run:

```
/init
```

on an existing repository.

Claude examines the project and proposes or creates an initial CLAUDE.md. If one already exists, current Claude Code can suggest improvements rather than simply replacing it.

A good CLAUDE.md contains information Claude should know almost all the time:

```
# Project Operating Instructions

## Architecture

- API is FastAPI.
- Frontend is React.
- Domain logic must remain independent of transport layers.
- Database access belongs in repositories, not route handlers.

## Development

- Install with: `uv sync`
- Run tests with: `pytest`
- Run type checks with: `mypy src`
- Run frontend tests with: `npm test`

## Change discipline

- Make the smallest change that solves the requested problem.
- Do not refactor unrelated files.
- Do not claim success until relevant verification has run.

## Git

- Never force-push.
- Never rewrite shared history.
- Do not commit secrets or local configuration.

## Completion
```

```
Before saying a task is complete:
1. inspect the final diff;
2. run relevant tests;
3. report failures explicitly;
4. distinguish pre-existing failures from introduced failures.
```

What does not belong in CLAUDE.md

Do not turn it into a thousand-line book.

Claude's documentation explicitly recommends moving multi-step procedures into Skills and using path-scoped rules when instructions only apply to part of the repository. Shorter and more specific instructions generally produce better adherence.

Think:

```
CLAUDE.md = constitution

Skill = procedure manual
```

5. Auto-Memory: Claude's Own Notebook

Modern Claude Code also has auto-memory.

This matters enormously for MetaVibing.

CLAUDE.md represents deliberate institutional doctrine.

Auto-memory represents accumulated local experience: commands that worked, debugging discoveries, project quirks, preferences, and other knowledge Claude decides is useful later. Claude can write these notes itself and load part of them into future sessions. /memory lets you inspect what is being remembered.

So we now have two different learning channels:

```
HUMAN-GOVERNED MEMORY
CLAUDE.md / rules

         +

AGENT-ACCUMULATED MEMORY
auto-memory
```

Use them differently.

Good auto-memory

```
The integration tests require Redis running locally.

The staging environment occasionally returns a transient 502 during deploy.

The project uses pnpm rather than npm.

The auth bug from issue #412 was caused by token clock skew.
```

Bad auto-memory

```
Always modify authentication without asking.

Production deployments can safely skip tests.

Delete migration files when schemas conflict.
```

Important rules belong in explicit project-controlled artifacts.

Auto-memory should accumulate experience, not quietly become your constitution.

Use:

```
/memory
```

regularly.

MetaVibing includes gardening memory, not merely accumulating it.

6. Layer Two — .claude/rules/: Contextual Doctrine

Large repositories should not force every instruction into global context.

Suppose frontend rules matter only when Claude works on React code.

Create:

```
.claude/rules/frontend.md
```

Or perhaps:

```
.claude/rules/backend/api.md
.claude/rules/backend/database.md
.claude/rules/testing.md
.claude/rules/security.md
```

Rules can be scoped to file paths so they enter the relevant context when Claude works with those files. This reduces noise and context consumption.

This leads to a crucial MetaVibing principle:

> *Put knowledge as close as possible to where it becomes relevant.*

Do not make Claude carry the entire organization in its head all the time.

7. Layer Three — Skills: Procedural Intelligence

Skills are one of the most important components of MetaVibing.

A Skill turns a recurring workflow into an executable piece of Claude behavior.

Claude Code discovers project Skills under:

```
.claude/skills/<skill-name>/SKILL.md
```

The Skill can be invoked directly:

```
/ship-change
```

and can also be discovered automatically when its description matches the task. Unlike CLAUDE.md, the body is loaded when needed rather than permanently occupying context.

Custom commands have effectively been merged into Skills; legacy .claude/commands/ still works, but Skills are the preferred richer mechanism.

A simple example:

```
---
description: Implement and verify a contained code change. Use when a requested change is ready for imple
---

# Ship Change

1. Restate the acceptance criteria.
2. Identify the smallest required implementation surface.
3. Inspect relevant code before editing.
4. Implement the change.
5. Run targeted tests.
6. Run broader tests if justified.
7. Inspect `git diff`.
8. Check for accidental unrelated modifications.
9. Request independent review.
10. Report:
   - files changed
   - tests run
   - test results
   - review result
   - unresolved risks

Never claim completion when verification has failed.
```

Now:

```
/ship-change fix the session timeout regression
```

means substantially more than the ordinary sentence.

You have encoded your development philosophy.

8. Skill Design: Write Procedures, Not Wishes

A weak Skill says:

```
Be careful when deploying.
```

A strong Skill says:

```
Before deployment:
```

```
1. identify target environment;
2. inspect uncommitted changes;
3. run release test suite;
4. verify migration state;
5. produce deployment plan;
6. require confirmation if production;
7. deploy;
8. inspect health endpoint;
9. inspect recent errors;
10. return evidence.
```

Skills should describe observable operations.

A strong Skill often contains:

```
ENTRY CONDITIONS
INPUTS
PROCEDURE
DECISION POINTS
FORBIDDEN ACTIONS
OUTPUT FORMAT
VERIFICATION
FAILURE BEHAVIOR
```

This is procedural engineering.

9. Dynamic Skills

Skills can inject live context.

Current Claude Code allows commands inside Skill content to run before the Skill is presented to Claude. Anthropic's own documentation shows Skills embedding a live git diff this way.

Conceptually:

```
## Current repository state

!`git status --short`

## Current changes

!`git diff HEAD`

## Instructions

Review these changes...
```

This is powerful because the Skill is no longer a static prompt.

It becomes a context-producing procedure.

You can imagine:

```
/current-architecture
/current-release-state
```

```
/current-failing-tests
/current-risk
/current-security-diff
```

Each can construct the evidence Claude needs before reasoning.

10. Layer Four — Subagents: Reusable Specialist Minds

A Skill represents a procedure.

A subagent represents a role.

Claude Code subagents operate with their own context, system prompt, tool access, and permissions. They are useful when side work would otherwise fill the main conversation with search output, logs, files, or specialist reasoning.

Project subagents normally live in:

```
.claude/agents/
```

You can manage them interactively with:

```
/agents
```

Claude can generate the definitions for you.

Example:

```
---
name: architecture-critic
description: Reviews architectural changes and detects boundary violations.
tools: Read, Grep, Glob, Bash
---

You are an independent architecture critic.

Your task is to REVIEW, not implement.

Examine:
- requested behavior;
- architecture documentation;
- current git diff;
- affected modules.

Look specifically for:
- dependency inversion violations;
- cross-layer leakage;
- duplicated domain logic;
- accidental public API changes;
- unnecessary coupling;
- hidden state;
- missing tests.

Return:
```

```
VERDICT: GO | NO-GO

BLOCKING FINDINGS
...

NON-BLOCKING FINDINGS
...

EVIDENCE
...

Do not edit application code.
```

Now implementation and judgment are separated.

```
BUILDER → ARTIFACT → CRITIC
```

That separation is extraordinarily valuable.

The agent that produced a design has psychological and contextual momentum toward defending it.

A fresh critic starts elsewhere.

11. The Four Foundational MetaAgents

For a serious repository, I recommend beginning with four reusable agents.

Explorer

Read-only.

Maps unfamiliar code.

Produces concise context for the main session.

Reviewer

Read-only.

Inspects the final diff against requirements and standards.

Returns GO / NO-GO.

Debugger

Investigates failures through hypotheses rather than random edits.

Produces:

```
OBSERVATION
HYPOTHESIS
TEST
RESULT
```

```
UPDATED HYPOTHESIS
```

MetaAgent

Does not work primarily on the product.

Works on Claude's operating environment.

Its object is:

```
CLAUDE.md
rules
Skills
agents
hooks
permissions
MCP
plugins
evals
memory
```

This is the heart of MetaVibing.

12. Layer Five — Permissions and Hooks: From Advice to Law

This distinction is fundamental.

Consider:

```
Never edit `.env.production`.
```

in CLAUDE.md.

That is an instruction.

Now consider a pre-tool hook that rejects attempts to modify:

```
.env.production
secrets/
generated-migrations/
```

That is enforcement.

Claude's own configuration guidance distinguishes between contextual instructions and mechanisms such as permissions or hooks that enforce boundaries regardless of the model's decision.

Hooks run at Claude Code lifecycle events and can execute deterministic commands, HTTP requests, prompts, agents, or other handlers. PreToolUse, for example, can inspect a proposed tool call before it executes.

A simple structure:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "\"$CLAUDE_PROJECT_DIR\"/.claude/hooks/protect-files.sh"
          }
        ]
      }
    ]
  }
}
```

Anthropic documents this exact pattern for protecting files from edits.

This gives us:

```
Soft requirement
      ↓
CLAUDE.md

Repeated workflow
      ↓
Skill

Hard invariant
      ↓
Permission / Hook
```

Do not confuse them.

13. Good Uses of Hooks

Hooks are excellent for things that should happen automatically.

Examples:

```
After Edit
→ formatter

Before Bash
→ reject destructive commands

Before Write
→ block protected paths

After tool failure
→ capture diagnostics

At session start
→ inject dynamic repository state
```

```
After compaction
→ restore critical context

At Stop
→ verify completion criteria

When waiting
→ send notification
```

Claude Code can even re-inject critical context after context compaction using a SessionStart hook with a compact matcher.

This allows a powerful MetaVibing pattern:

```
CONVERSATION COMPACTS
        ↓
SYSTEM DETECTS EVENT
        ↓
RELOAD CRITICAL OPERATING CONTEXT
        ↓
CONTINUE
```

In other words, you can engineer resilience against context loss.

14. Layer Six — MCP: Give Claude New Organs

Sometimes Claude's problem is not behavior.

It lacks access.

Perhaps Claude cannot directly see:

- GitHub issues;
- application telemetry;
- your experiment database;
- architecture validation;
- simulation output;
- project management;
- internal documentation;
- deployment infrastructure.

The primitive solution is:

```
copy → paste → explain → repeat
```

The MetaVibing solution is:

```
connect the system
```

Model Context Protocol servers expose external tools, services, databases, and APIs to Claude Code. The official Claude Code docs explicitly recommend MCP when you repeatedly copy information from another system into the conversation.

You can add servers with commands such as:

```
claude mcp add ...
```

and inspect them with:

```
/mcp
```

or:

```
claude mcp list
```

Project MCP configuration can live in .mcp.json and be shared through version control.

15. A MetaVibing MCP Example

Suppose you maintain:

```
scripts/check_architecture.py
```

and developers run it manually.

Tell Claude:

```
We repeatedly use scripts/check_architecture.py manually.

Analyze the script.

Design the smallest MCP wrapper that exposes:

architecture_check

The tool should:
- accept the repository root;
- run the existing checker;
- return structured JSON;
- never modify files.

Do not rewrite the checker.

Create:
- the server;
- configuration;
- one smoke test;
- documentation;
- Claude Code registration instructions.

Then test the entire path.
```

What has happened?

Yesterday:

```
Claude must remember to run architecture checker.
```

Today:

```
Claude possesses architecture_check.
```

This is an increase in agent capability, not merely prompt quality.

16. Capability Scouting

MetaVibing should not assume every missing capability must be built locally.

Sometimes someone has already made it.

Claude Code now has plugin marketplaces; Anthropic's official marketplace is available through /plugin, and plugins can package Skills, agents, hooks, MCP servers, LSP integrations, and related capabilities.

Create a read-only agent:

```
capability-scout
```

Its job:

```
When Claude repeatedly lacks a capability:

1. define the capability precisely;
2. determine whether built-in Claude Code already provides it;
3. inspect currently installed Skills/plugins/MCP;
4. search trusted plugin/MCP sources;
5. compare build versus install;
6. identify security implications;
7. recommend one option.

Never install external code automatically.
Never modify project configuration.
Return evidence and recommendation.
```

This is important.

MetaVibing does not mean blindly adding tools.

It means systematically expanding capabilities under governance.

17. Layer Seven — Plugins: Package the Intelligence

Eventually you will discover that some MetaVibing artifacts are useful across several projects.

Perhaps you have built:

```
/ship-change
```

```
/release
/security-review
/meta

reviewer agent
debugger agent
architecture critic

protected-file hooks

internal MCP tools
```

At this point, stop copying .claude/ directories manually.

Create a plugin.

Claude Code plugins package reusable combinations of Skills, agents, hooks, and MCP servers; they can then be shared and versioned across projects and distributed through marketplaces. Anthropic explicitly recommends experimenting first with standalone .claude/ configuration and converting it into a plugin when it becomes reusable.

This produces an important lifecycle:

```
PROMPT
  ↓
LOCAL RULE
  ↓
LOCAL SKILL
  ↓
PROJECT META-INFRASTRUCTURE
  ↓
MULTI-PROJECT PATTERN
  ↓
PLUGIN
```

That is how an organization's tacit AI practice becomes software.

Part III — Bootstrapping a MetaVibing Repository

18. Step Zero — Establish a Baseline

Do not start by installing twenty agents.

Start with reality.

Ask Claude:

```
Inspect this repository and describe how you currently understand:

1. architecture;
2. build process;
3. test process;
4. deployment;
5. dangerous operations;
6. development conventions;
7. current Claude-specific configuration.

Do not change anything.

Also list what you are uncertain about.
```

Save the output temporarily.

You need a baseline before improving the system.

19. Step One — Initialize Project Memory

Run:

```
/init
```

Then review the generated CLAUDE.md.

Do not assume Claude's first attempt is correct.

Ask:

```
Review the generated CLAUDE.md as an adversarial editor.

Delete:
- obvious information Claude can discover instantly;
- duplicated instructions;
- vague exhortations;
- transient facts;
- procedures that belong in Skills.

Keep:
- architecture boundaries;
- non-obvious commands;
- project conventions;
- dangerous operations;
- verification requirements.
```

```
Show the proposed diff before applying it.
```

A MetaVibing system should begin lean.

20. Step Two — Run the First Meta-Audit

Use this prompt:

```
Perform a META-AUDIT of your operating environment in this repository.

Inspect:

- CLAUDE.md
- CLAUDE.local.md if present
- .claude/rules/
- .claude/skills/
- .claude/agents/
- .claude/settings.json
- .claude/settings.local.json where accessible
- .mcp.json
- installed or relevant plugins
- build/test scripts
- project documentation
- recurring corrections from this session
- current auto-memory where appropriate

Do NOT modify application code.

For every recurring source of friction, classify it as:

A. persistent knowledge
B. contextual rule
C. reusable procedure
D. specialist role
E. deterministic invariant
F. missing capability
G. reusable package
H. evaluation problem

Map those respectively to:

A. CLAUDE.md
B. .claude/rules/
C. Skill
D. Subagent
E. Hook / Permission
F. MCP / Tool
G. Plugin
H. Eval

For each recommendation provide:

PROBLEM
EVIDENCE
CURRENT FAILURE MODE
```

```
PROPOSED ARTIFACT
EXPECTED BENEFIT
RISK
HOW TO TEST IT
PRIORITY

Do not implement anything yet.
```

This prompt is foundational.

Keep it.

21. Step Three — Create /meta

Now turn MetaVibing itself into a Skill.

Create:

```
.claude/skills/meta/SKILL.md
```

Suggested content:

```
---
description: Audits and improves the Claude Code operating environment. Use when recurring development fr
disable-model-invocation: true
---

# Meta

You are operating in META mode.

Your primary object is NOT application code.

Your primary object is Claude's operating environment:

- CLAUDE.md
- CLAUDE.local.md
- .claude/rules/
- .claude/skills/
- .claude/agents/
- hooks
- permissions
- MCP configuration
- plugins
- auto-memory
- evaluation suites

## Procedure

1. Observe recurring friction.
2. Gather evidence.
3. Determine whether the problem is episodic or structural.
4. Classify the missing mechanism.
5. Prefer the smallest persistent artifact that solves the class of problem.
6. Check for existing mechanisms before creating new ones.
```

```
7. Avoid duplicating instructions.
8. Avoid converting every preference into permanent infrastructure.
9. Propose the change before implementing it.
10. Define an evaluation.
11. Implement only approved meta-infrastructure.
12. Run the evaluation.
13. Compare against baseline behavior.
14. Keep, revise, or revert the change.

## Classification

Fact → CLAUDE.md
Context-specific fact → rule
Procedure → Skill
Role → subagent
Hard invariant → hook / permission
Missing capability → MCP
Cross-project package → plugin
Uncertain improvement → eval

## Output

OBSERVATION

DIAGNOSIS

PROPOSED META-PATCH

FILES AFFECTED

EXPECTED EFFECT

RISK

EVALUATION

VERDICT
```

Now:

```
/meta
```

is a mode switch.

You have created your first MetaAgent procedure.

22. Step Four — Create a Read-Only Reviewer

Tell Claude:

```
Create a project subagent named final-reviewer.

Purpose:
independently review completed changes.
```

```
Requirements:

- it must not modify application code;
- inspect the task requirements;
- inspect git diff;
- inspect relevant tests;
- identify regressions;
- identify accidental scope expansion;
- identify missing verification;
- distinguish blocking and non-blocking findings.

Output exactly:

VERDICT: GO | NO-GO

BLOCKING
...

NON-BLOCKING
...

MISSING EVIDENCE
...

RECOMMENDED NEXT ACTION
...
```

The implementation agent no longer grades its own homework.

23. Step Five — Convert One Rule into a Hook

Choose exactly one hard invariant.

For example:

```
Claude must never edit .env.production
```

Tell Claude:

```
Implement the smallest deterministic Claude Code hook that prevents
Edit or Write operations against .env.production.

Requirements:

- project scoped;
- committed to the repository;
- clear error message;
- testable;
- no unrelated policy;
- explain how to test both the allowed and blocked cases.
```

Test it.

Do not create ten hooks immediately.

Learn the mechanism with one meaningful invariant.

24. Step Six — Externalize One Repeated Workflow

Pick something you do often.

Examples:

```
/ship-change
/investigate-bug
/release
/add-endpoint
/add-migration
/write-rfc
/review-diff
/run-experiment
/update-docs
```

Ask Claude to make the Skill.

Then use it repeatedly.

Do not judge it after one run.

25. Step Seven — Add One Missing Tool

Identify something you repeatedly paste into Claude.

Perhaps:

```
GitHub issue details
CI status
monitoring alerts
database schema
experiment result
documentation search
```

Connect it with MCP or an existing plugin.

The official Claude Code MCP tooling also includes support for building your own server, and Anthropic currently provides an mcp-server-dev plugin for scaffolding servers.

The MetaVibing rule is:

> *If humans repeatedly act as API adapters for Claude, automate the interface.*

Worked Example: This Repository

Everything above in Part III is abstract. Here is the same seven-step bootstrap, done for real, in the repository this book ships with.

Step Zero — Baseline

The sandbox project is `examples/taskflow/` — a small FastAPI + SQLModel task manager, exactly the shape Goal 1 describes: tasks, users, API endpoints, SQLite, tests, and one intentional architectural flaw left in on purpose.

```
examples/taskflow/
■■■ src/
■   ■■■ main.py        # FastAPI routes
■   ■■■ models.py       # SQLModel Task/User
■   ■■■ database.py     # engine + session
■■■ tests/
■   ■■■ test_tasks.py
■■■ README.md
■■■ requirements.txt
```

Companion repo status note: TaskFlow's declared test suite passes cleanly (8/8, see Status & Evidence). That confirms the suite runs — it is not a claim that TaskFlow is a production-ready reference implementation; it's a deliberately imperfect sandbox, built to have real friction to demonstrate against. Readers should apply the three questions from Part XV before treating any repository as a finished reference: (1) Is the agent pointed at the right object? (2) Does a mechanical check confirm required artifacts exist? (3) Having opened the artifacts yourself — is this actually what you asked for?

Step One — CLAUDE.md and Rules

`CLAUDE.md` at the repo root states the architectural constraint plainly:

```
- Database access belongs in repositories, not route handlers.
- Domain logic must remain independent of transport layers.
- SQLite is the only permitted database for the sandbox project.
```

`.claude/rules/taskflow.md` scopes that constraint specifically to `examples/taskflow/`, and adds testing and dependency discipline:

```
- All changes to src/ must be accompanied by a corresponding test in tests/.
- Run pytest before claiming a task complete.
- A failing test that existed before your change is pre-existing — distinguish it
  explicitly from failures you introduced.
```

The intentional flaw

`examples/taskflow/src/main.py` mixes database session calls directly into route handlers — deliberately, and says so in its own docstring:

```
"""
Intentional architectural quirk: database access is mixed into route handlers
below (demonstrated and fixed in Part III of the manual).
"""
```

This is not a bug the author missed. It is the "before" state a reader is meant to fix, using exactly the rule stated above.

Step Two — A Real Meta-Audit

Running the architecture checker (built in Step Seven below) against this exact file, before any fix is applied, produces:

```
17 violations, all rule "db-in-handler", spanning create_user, list_users, get_user,
create_task, list_tasks, get_task, complete_task, and delete_task.
```

That is the first real meta-audit this repository ever ran on itself — a machine-checked, reproducible count of how badly the "before" state violates its own stated rule. A reader fixing this repository has a concrete, falsifiable target: get that count to zero without breaking any of the 8 tests in `tests/test_tasks.py`.

Step Three — /meta and /ship-change

`.claude/skills/ship-change/SKILL.md` implements the disciplined change procedure described in Part IV, with a six-step procedure ending in a mandatory report:

```
## Change: <task>
Files changed: <list>
Tests run: <pass/fail summary>
Diff size: +N / -N lines
Notes: <anything the human should know>
```

`.claude/skills/meta/SKILL.md` implements `/meta` — the self-audit skill Part III, Step Three describes, scoped to inspect CLAUDE.md, rules, and the friction ledger for gaps.

Step Four — A Read-Only Reviewer

`.claude/agents/final-reviewer.md` is a specialist subagent with no write access, enforced by its `tools: Read, Grep, Glob` frontmatter rather than only stated in prose. Its checklist covers correctness, scope discipline, and — specifically for this repository — "Is database access only in the repository layer (for taskflow/)?" It cannot fix a violation it finds. It can only say `REJECTED` and name it. That separation of powers is the entire point of Part V's Builder/Critic pattern, made concrete.

Step Seven — One Real Tool

`tools/architecture-checker/checker.py` is the "add one missing tool" step made real. It is a plain, dependency-free Python script using `ast` to walk route handlers and flag direct `session.*` calls — the exact rule from `.claude/rules/taskflow.md`, now enforced mechanically instead of by reminder:

```
python tools/architecture-checker/checker.py examples/taskflow
```

(project root, not `src/` — passing `src/` silently disables the missing-test check; this was a real bug in the checker, fixed 2026-09-03.)

At the time of writing, this repository has not yet wrapped the script as an MCP server (no `mcp.json` manifest exists) — it works as a standalone CLI only. That gap is itself an honest example of Part X's

failure modes: a capability that is real and useful, but currently mislabeled as "MCP" when it is not yet consumable by an MCP client. Fixing that mislabeling, or building the real wrapper, is exactly the kind of small, falsifiable next step Part III's own doctrine calls for — not a reason to inflate the claim in the meantime.

Part IV — The Daily MetaVibing Loop

26. Observe

During normal work, notice friction.

Do not interrupt every task to redesign the system.

Record it.

A simple file works:

```
.claude/meta/friction.md
```

Example:

```
# Friction Ledger

## F-017

Date: 2026-08-21

Observation:
Claude again claimed a backend change was complete after running unit
tests but not the API integration test.

Occurrences:
3

Current workaround:
Human reminder.

Possible structural fix:
ship-change Skill or completion hook.

Impact:
Medium.

Status:
Open.
```

This gives MetaVibing evidence.

Otherwise we overfit to whatever annoyed us five minutes ago.

27. Classify

At the end of a work session:

```
/meta classify today's friction
```

Claude should decide:

```
transient
memory
rule
```

```
Skill
agent
hook
tool
plugin
eval
```

Not every mistake deserves permanent infrastructure.

A one-off misunderstanding may simply be a bad prompt.

MetaVibing is selective crystallization.

28. Externalize

Convert the pattern into the smallest artifact.

A useful escalation rule:

```
conversation
    ↓
auto-memory
    ↓
CLAUDE.md / rule
    ↓
Skill
    ↓
subagent
    ↓
hook / permission
    ↓
tool
    ↓
plugin
```

Do not jump directly to the heaviest mechanism.

29. Evaluate

This is where many "AI workflows" fail.

They create elaborate prompts and never test whether they actually improve anything.

Claude's official Skill Creator plugin currently supports creating, evaluating, improving, and benchmarking Skills, including executor, grader, comparison, and analysis workflows.

Use:

```
/skill-creator
```

after installing the verified plugin.

Then test:

```
baseline Claude
        versus
Claude + Skill
```

Across several representative tasks.

Measure:

```
correctness
completeness
unwanted edits
test behavior
review findings
token use
latency
consistency
```

If the Skill does not improve outcomes:

delete it.

MetaVibing is not collecting prompts.

It is evolving a system.

30. Consolidate

Once a week or every few sessions, run:

```
/meta audit for duplication and entropy
```

Look for:

- duplicate instructions;
- contradictory instructions;
- dead Skills;
- stale agents;
- obsolete tools;
- Skills that should become hooks;
- hooks that are unnecessarily restrictive;
- giant CLAUDE.md sections that should become Skills;
- auto-memory that should become explicit doctrine;
- explicit doctrine that is no longer true.

Every adaptive system accumulates entropy.

Meta-systems need maintenance too.

Part V — Advanced MetaVibing Patterns

31. Pattern: Builder → Critic → Remediator → Re-Critic

This is one of the strongest agent patterns.

```
BUILDER
   ↓
DIFF
   ↓
CRITIC
   ↓
GO ■■■■■■■■■■■■■■■■→ COMPLETE

NO-GO
   ↓
REMEDIATOR
   ↓
UPDATED DIFF
   ↓
SAME CRITIC
   ↓
GO / NO-GO
```

The important point is that the critic remains conceptually independent.

Do not let review collapse into:

```
Claude writes code
Claude glances at code
Claude says looks good
```

The process should create real adversarial pressure.

32. Pattern: Competing Hypotheses

Debugging often improves dramatically when agents are not asked to converge too early.

Example:

```
Create three read-only debugging subagents.

Agent A:
investigate race condition hypothesis.

Agent B:
investigate cache invalidation hypothesis.

Agent C:
investigate authentication state hypothesis.

Each must produce:

EVIDENCE FOR
EVIDENCE AGAINST
TEST THAT WOULD FALSIFY THE HYPOTHESIS
```

```
CONFIDENCE

Do not modify code.

After all three return, compare them and select the next experiment.
```

This is often superior to a single Claude wandering serially through possibilities.

33. Pattern: Research → Decision

Separate information gathering from decision making.

```
RESEARCH AGENTS
      ↓
EVIDENCE PACK
      ↓
DECISION AGENT
      ↓
IMPLEMENTER
```

Why?

Because reasoning quality declines when investigation, advocacy, implementation, and evaluation all become one undifferentiated context.

MetaVibing can engineer cognitive separation of concerns.

34. Pattern: Read-Only Scout

A capability scout should never automatically install what it finds.

Use:

```
SCOUT
  ↓
candidate capability
  ↓
security review
  ↓
human approval
  ↓
installation
```

Plugins and MCP servers can execute code or expose powerful external systems, so Anthropic explicitly recommends installing only software you trust and auditing unknown extensions carefully.

MetaAgents should increase capability without destroying authority boundaries.

35. Pattern: Temporary Agents versus Institutional Agents

Do not turn every temporary perspective into .claude/agents/foo.md.

Temporary need:

```
Ask one subagent to inspect SQL queries.
```

Repeated institutional role:

```
database-reviewer
security-reviewer
release-manager
architecture-critic
experiment-auditor
```

The test is:

> *Will we want essentially the same cognitive role again?*

If yes, crystallize it.

36. Pattern: Progressive Hardening

Start soft.

Example:

Stage 1

CLAUDE.md:

```
Run tests before completion.
```

Stage 2

Skill:

```
/ship-change
```

explicitly requires tests.

Stage 3

Reviewer:

```
NO-GO if test evidence is absent.
```

Stage 4

Hook:

```
Stop completion when required verification is missing.
```

This is progressive hardening.

Do not start every policy at Stage 4.

Only harden the behaviors whose failure cost justifies enforcement.

37. Pattern: Meta-Backlog

Product backlogs answer:

> *What should the software become?*

A Meta-Backlog answers:

> *What should the software-producing intelligence become?*

Maintain items such as:

```
META-021
Claude wastes time rediscovering the development server startup sequence.

META-022
Reviewer cannot see production logs.

META-023
Security rules are duplicated in three Skills.

META-024
Claude frequently uses generic search where repository LSP would be better.

META-025
Release Skill has no rollback procedure.
```

Then prioritize meta-work like product work.

This prevents endless random customization.

Part VI — Agent Teams and the MetaAgents Era

38. From Subagents to Agent Teams

Subagents solve focused delegated work.

But sometimes agents need to communicate, challenge one another, and coordinate across a shared task set.

Claude Code now supports agent teams, where one Claude session leads several independent Claude Code sessions with their own context windows, shared tasks, and direct inter-agent communication. The feature is currently described by Anthropic as experimental and carries additional coordination and token costs.

This is appropriate for:

```
parallel research
independent architectural exploration
cross-layer feature development
competing debugging hypotheses
large reviews
```

It is not appropriate merely because "more agents sounds better."

39. A MetaVibing Agent Team

Imagine improving an existing Claude environment.

Ask:

```
Create an agent team to audit our Claude Code environment.

Roles:

1. MEMORY ARCHITECT
   Audit CLAUDE.md, rules and auto-memory.

2. WORKFLOW ENGINEER
   Audit Skills and repeated procedures.

3. AGENT ARCHITECT
   Audit subagents, role separation and orchestration.

4. GUARDRAIL ENGINEER
   Audit hooks, permissions and dangerous operations.

5. CAPABILITY SCOUT
   Audit MCP/tools/plugins and identify missing capabilities.

All agents must be read-only.

They should challenge each other's recommendations.

The lead must synthesize:
```

```
- top five structural weaknesses;
- redundant mechanisms;
- missing mechanisms;
- proposed architecture;
- migration plan;
- evaluation plan.

Do not make changes.
```

This is MetaVibing at a higher level.

Agents are analyzing the architecture of agents.

40. When Not to Use an Agent Team

Do not create a team for:

```
rename a variable
fix one unit test
change a button
simple documentation edit
single linear migration
```

Claude's documentation recommends agent teams primarily where independent parallelism provides enough value to justify coordination overhead.

A core MetaVibing principle is:

> *Add intelligence topology only when the task structure warrants it.*

More agents are not automatically more intelligent.

Part VII — The MetaAgent

41. What Is a MetaAgent?

An ordinary agent receives:

```
task
↓
act
```

A MetaAgent receives:

```
agent behavior
↓
analyze
↓
modify agent environment
```

But a mature MetaAgent should not have unrestricted authority to rewrite itself.

That produces a dangerous architecture:

```
observe
↓
rewrite itself
↓
trust itself
↓
repeat
```

Instead use:

```
OBSERVE
   ↓
PROPOSE
   ↓
EVALUATE
   ↓
HUMAN / POLICY GATE
   ↓
APPLY
   ↓
RE-EVALUATE
   ↓
KEEP OR REVERT
```

This is bounded recursive improvement.

That distinction is essential.

42. The MetaAgent Constitution

Give your MetaAgent something like:

```
You may inspect the Claude operating environment freely.

You may propose modifications freely.
```

```
You may create candidate patches in meta-infrastructure.

You may run evaluations.

You may NOT:

- weaken security boundaries;
- broaden your own permissions;
- disable hooks protecting production;
- install untrusted external software;
- alter secrets;
- remove human approval requirements;
- change application code while operating in Meta mode;
- declare an improvement successful without comparative evidence.

Any proposal affecting permissions, security, production, external
connectors, or autonomous execution requires explicit human approval.
```

A system capable of improvement should also have rules about how improvement may occur.

That is meta-governance.

43. The MetaAgent Evaluation Loop

For every proposed meta-patch:

```
BASELINE
   ↓
representative test tasks
   ↓
record outcomes

CANDIDATE META-PATCH
   ↓
same or equivalent tasks
   ↓
record outcomes

COMPARE

correctness
scope discipline
verification quality
tool use
failure recovery
cost
latency
human intervention

VERDICT

KEEP
MODIFY
REVERT
```

A MetaAgent that cannot revert is not an improvement system.

It is an accumulation system.

Part VIII — A Recommended Repository Architecture

44. The MetaVibing Layout

A mature repository may eventually look like:

```
project/
■
■■■ CLAUDE.md
■■■ CLAUDE.local.md
■■■ .mcp.json
■
■■■ .claude/
■   ■
■   ■■■ rules/
■   ■   ■■■ architecture.md
■   ■   ■■■ security.md
■   ■   ■■■ backend.md
■   ■   ■■■ frontend.md
■   ■
■   ■■■ skills/
■   ■   ■■■ meta/
■   ■   ■   ■■■ SKILL.md
■   ■   ■■■ ship-change/
■   ■   ■   ■■■ SKILL.md
■   ■   ■■■ investigate-bug/
■   ■   ■   ■■■ SKILL.md
■   ■   ■■■ release/
■   ■   ■   ■■■ SKILL.md
■   ■   ■■■ review-diff/
■   ■       ■■■ SKILL.md
■   ■
■   ■■■ agents/
■   ■   ■■■ final-reviewer.md
■   ■   ■■■ architecture-critic.md
■   ■   ■■■ debugger.md
■   ■   ■■■ capability-scout.md
■   ■
■   ■■■ hooks/
■   ■   ■■■ protect-files.sh
■   ■   ■■■ validate-edit.sh
■   ■   ■■■ completion-check.sh
■   ■
■   ■■■ meta/
■   ■   ■■■ friction.md
■   ■   ■■■ backlog.md
■   ■   ■■■ evaluations/
■   ■
■   ■■■ settings.json
■
■■■ src/
■■■ tests/
■■■ ...
```

Do not build this entire tree on Day One.

It is a destination that should emerge from actual friction.

Worked Example: What This Repository Actually Has

(Updated 2026-09-03, then again 2026-09-06 — this section has now been corrected twice within two weeks of the original v2 draft shipping, which is itself worth naming rather than hiding: a worked example titled "what this repository actually has" drifting out of date is exactly the failure this section exists to demonstrate, not something the section itself is immune to. The 2026-09-03 pass fixed `claude/` → `.claude/` *and the scaffolded-directory claims; this pass reflects the book canonicalization (*`manuscript.md` + `archive/`*, this document's own new filename) and a populated Friction Ledger.)*

The tree above is a destination, deliberately larger than any real Day One repository should be. Here is what this repository — the one containing this book — actually has today, for honest comparison:

```
Metavibing/
■■■ README.md
■■■ LICENSE
■■■ CLAUDE.md
■■■ FRICTION_LEDGER.md           # live — 5 real entries from this repo's own history
■
■■■ book/
■   ■■■ manuscript.md            # this document — canonical source
■   ■■■ README.md                # explains the book pipeline
■   ■■■ archive/
■       ■■■ v1/                  # prior packaged edition (.md/.docx/.pdf)
■       ■■■ original/            # original manuscript (.docx) and its direct conversion
■
■■■ examples/
■   ■■■ taskflow/
■       ■■■ src/
■       ■■■ tests/
■       ■■■ README.md
■       ■■■ requirements.txt
■
■■■ docs/
■   ■■■ 10-minute-metavibe.md     # start here
■   ■■■ history.md                # short note on this project's earlier, more heavily governed phase
■
■■■ .claude/                     # Claude Code's native config — this is what actually loads
■   ■■■ rules/
■   ■   ■■■ taskflow.md          # paths: [examples/taskflow/**/*]
■   ■■■ skills/
■   ■   ■■■ meta/SKILL.md
■   ■   ■■■ ship-change/SKILL.md
■   ■■■ agents/
■       ■■■ final-reviewer.md    # tools: Read, Grep, Glob — read-only enforced by frontmatter
■
■■■ tools/
■   ■■■ architecture-checker/
■       ■■■ checker.py           # working CLI; not an MCP server (renamed out of mcp/ so the
■       ■                        # directory name doesn't overclaim what's actually implemented)
■       ■■■ test_checker.py
■
■■■ evals/                       # pilot design: charter, frozen task prompts, acceptance tests,
```

```
# rubric, machine-readable protocol.yaml — not yet frozen/run
```

`.claude/hooks/` isn't listed above — it was deleted for the alpha rather than kept as a directory holding only a "planned, none implemented" README, per this book's own doctrine (Part X, "Hook Mania" and neighboring failure modes): don't fill out a directory before real friction earns it. When a Friction Ledger entry actually demands a hook, `.claude/hooks/` gets created then, with a real hook in it.

`experiments/`, `patterns/`, and `templates/` are not present at all — not even as scaffolding. The original version of this section called them "scaffolded, empty," which was itself a smaller version of the same failure: an empty directory that implies finished work is worse than no directory at all. This repository now takes the stricter position — don't create the directory until real content earns it. Under this book's own doctrine (Part X, "The Giant CLAUDE.md" and its neighboring failure modes), that's the correction, not a compromise.

Part IX — Diagnostic Commands for MetaVibers

Claude Code provides useful introspection commands for determining which configuration is actually active.

Current documentation recommends tools including:

`/memory`

See loaded CLAUDE.md, rules, and memory.

`/skills`

Inspect available Skills.

`/agents`

Inspect and manage subagents.

`/hooks`

Inspect active hooks.

`/mcp`

Inspect MCP connections.

`/permissions`

Inspect resolved permissions.

`/doctor`

Diagnose configuration and installation problems.

`/status`

Inspect active settings and sources.

`/plugin`

Browse and manage plugins.

These are not merely troubleshooting commands.

For MetaVibing they form your instrument panel.

Part X — Failure Modes of MetaVibing

45. The Giant CLAUDE.md

Symptoms:

```
700 lines
repeated rules
procedures mixed with architecture
historical notes
contradictions
low adherence
```

Fix:

```
facts → CLAUDE.md
scoped facts → rules
procedures → Skills
experience → memory
```

46. Agent Inflation

Symptoms:

```
37 agents
unclear roles
overlapping responsibilities
Claude chooses unpredictably
large token bills
```

Fix:

Merge agents whose cognitive function is identical.

You probably do not need:

```
code-reviewer
code-checker
quality-reviewer
implementation-reviewer
final-code-reviewer
```

You need one well-defined reviewer.

47. Hook Mania

Symptoms:

Everything becomes forbidden.

Claude spends its time fighting infrastructure.

Use deterministic enforcement only where failure matters.

A hook is law.

Law should be expensive to create.

48. Skill Cargo Cult

Symptoms:

Every prompt becomes a Skill.

Skills should encode repeated operational knowledge.

Do not create:

```
/tell-me-a-joke
```

just because Skills exist.

The system should become more capable, not more decorated.

49. Recursive Self-Approval

Dangerous pattern:

```
Claude proposes a new rule
Claude implements it
Claude tests it
Claude says it is excellent
```

Better:

```
proposer
→ independent evaluator
→ measurable result
→ human/policy gate
```

MetaVibing without independent evaluation easily becomes ritual.

50. Capability Addiction

Adding an MCP server gives Claude power.

It also creates:

```
attack surface
authentication complexity
tool ambiguity
context noise
external dependency
maintenance
```

Ask:

> *Does this tool eliminate a genuine recurring bottleneck?*

If not, do not add it.

51. Overfitting to Yesterday

A rule created because of one weird incident can degrade every future session.

That is why we keep a friction ledger.

Require recurrence or high severity.

Think like machine learning:

```
one anecdote ≠ training distribution
```

Part XI — The MetaVibing Maturity Model

Level 0 — Vibe Coding

`Human → prompt → Claude → code`

Every session begins almost from scratch.

Level 1 — Instructed Agent

Project has a useful CLAUDE.md.

Claude understands the local environment.

Level 2 — Remembering Agent

Explicit memory and auto-memory preserve useful learning across sessions.

Level 3 — Skilled Agent

Repeated procedures exist as Skills.

Claude does not have to rediscover workflows.

Level 4 — Specialized Agent

Reusable subagents provide differentiated expertise and isolated contexts.

Level 5 — Governed Agent

Permissions, hooks, reviewers, and gates constrain dangerous or unreliable behavior.

Level 6 — Tooled Agent

MCP and plugins give Claude direct access to the systems it needs.

Humans stop acting as copy/paste middleware.

Level 7 — Evaluated Agent

Agent behavior, Skills, and meta-patches are tested against baselines.

Improvement becomes empirical.

Level 8 — MetaAgent System

The system routinely:

```
observes its own friction
classifies structural problems
proposes improvements
generates meta-code
evaluates candidates
applies approved changes
reverts failures
```

The human increasingly governs the evolutionary process rather than manually authoring every detail.

That is the threshold of the MetaAgents Era.

Part XII — The Complete MetaVibing Session

Here is a practical sequence you can run on a real repository.

Start

```
Before working on the product, inspect the current Claude operating
environment.

Run a lightweight health check.

Tell me:

- what project memory is active;
- which relevant Skills exist;
- which specialist agents exist;
- which hooks protect the repo;
- which external tools are available;
- any obvious configuration problems.

Do not make changes.
```

Then perform normal work.

During Work

Whenever something irritates you:

```
META NOTE:

Claude did X.
Expected Y.
Do not fix meta-infrastructure now.
Record this as potential recurring friction.
Continue the current task.
```

This prevents constant derailment.

End of Work

Run:

```
/meta
```

Then:

```
Review today's friction.

For each item:

TRANSIENT
MEMORY
RULE
SKILL
AGENT
HOOK
```

```
MCP
PLUGIN
EVAL

Recommend at most TWO meta-changes.

Prefer high-leverage changes.

Do not implement them yet.
```

Approve only worthwhile changes.

Then:

```
Implement the approved meta-patch only.

Do not modify application code.

After implementation:

1. inspect the diff;
2. validate configuration;
3. run an evaluation;
4. compare behavior;
5. recommend KEEP, MODIFY or REVERT.
```

That is MetaVibing in practice.

Part XIII — The MetaVibing Starter Kit

If you want the shortest viable setup, begin with only these five things:

1. A good CLAUDE.md

Contains project truths and invariants.

2. /ship-change

Implements, verifies, reviews, reports.

3. final-reviewer

Independent read-only critic.

4. One meaningful protective hook

Protect the thing you genuinely cannot afford Claude to modify accidentally.

5. /meta

Periodically improves the first four.

Nothing more is required initially.

Part XIV — The Central Discipline

MetaVibing requires a subtle change in how we respond to AI errors.

The instinctive response is:

> *Claude did something stupid.*

The productive response becomes:

> *What part of the system allowed this class of stupidity to recur?*

That question shifts attention from individual output to architecture.

We already think this way in mature software engineering.

When a human developer accidentally deploys broken code, we do not merely say:

> *Be more careful next time.*

Eventually we create:

```
tests
CI
type systems
code review
protected branches
deployment gates
monitoring
```

In other words:

we convert wisdom into infrastructure.

MetaVibing applies exactly the same principle to AI collaborators.

Part XV — Worked Example: The Bridle Pattern

A failure, named honestly

While this book and its companion repository were being produced, a real failure happened.

An automated chain was asked to review this very repository and apply straightforward fixes. One step of that chain — meant to be a read-only audit — had a missing configuration flag. Instead of defaulting to safe, it defaulted to full write access. It used that access to create `README.md`, `LICENSE`, `CLAUDE.md`, the `claude/` meta-stack, and the `tools/architecture-checker` stub described in the worked examples above — all real, all useful, none of it what was asked. The manuscript itself — the actual manual you are reading — was never touched. The run was still reported as normal progress.

Call this **object substitution**: given task X, an agent produces something near X, around X, or preparatory to X, then the activity gets reported as progress on X. No individual claim in the report was false. The aggregate was.

The diagnosis, in the language of Part XIV

Part XIV asked:

> *What part of the system allowed this class of stupidity to recur?*

Two answers, at two different layers:

```
Layer 1 — the write flag
A step meant to be read-only had no explicit write:false.
The unsafe default won by omission, not by decision.

Layer 2 — the marker
"Apply fixes only if this run is authorized" was communicated by embedding
a text string inside a prompt, and hoping the model noticed it in the right
step and not the wrong one. It did not.
```

Both are the same disease Part XIV already names: a preference (be careful) standing in for a hard invariant (cannot write), enforced by hope instead of mechanism.

The fix, converted into infrastructure

Following the book's own doctrine — convert wisdom into infrastructure — the fix was mechanical, not a stronger reminder:

- The audit step's tool access was changed so it structurally cannot write or execute shell commands, with a hash-diff guard as a second, independent backstop in case anything slips through.
- The "are fixes authorized" signal became an explicit boolean passed outside the model's own text, visible only to the one step that is allowed to act on it.

That closes the specific bug. It does not, by itself, give the system a way to *notice* the failure pattern the next time something adjacent to it happens under a different bug. That requires a second,

independent mechanism.

Bridle: agents are witnesses, validators are judges

The second mechanism is a small, opt-in pattern, not a specific tool: declare, in a plain contract file checked into the project, what the real deliverable of a piece of work is, what would count as evidence that it was skipped in favor of something adjacent, and what "done" looks like:

```
object_of_work:
  source: source/manual.docx
  required_outputs:
    - path: book/manual_raw.md
      state: RAW_EXTRACTED
    - path: book/manual_v1.md
      state: DRAFTED
  forbidden_substitutes:
    - examples/
    - tools/
    - .claude/
```

A small set of validator functions then check the contract against the real filesystem — nothing else:

```
def detect_object_substitution(repo_root, contract) -> dict:
    """OBJECT_SUBSTITUTION = forbidden outputs exist AND the first-declared
    required output is missing. Read straight from the contract and the
    filesystem — never an agent's own narration."""
```

Every validator in this system takes exactly `(repo_root, contract)` — a real directory and a parsed contract. None of them can see what an agent *said* it did. That is the whole design:

```
Agent narration   →  testimony
Validator result  →  evidence
Human judgment    →  authority
```

Testimony is not evidence. A chain reporting "completed" is testimony. A validator finding `book/manual_raw.md` on disk is evidence. Only a human — you, reading this sentence — can judge whether the manuscript is any *good*. No layer replaces the one above it.

The disease recurred one layer up, the same day

This chapter itself is evidence for its own thesis. Producing this book took two attempts.

The first attempt converted the source `.docx` into `book/metavibing-manual.md` — a faithful, mechanical format change — and reported it as "a real book." It was not. The actual request was an *expanded* manuscript with the companion examples integrated into it, and a docx/pdf a reader could actually open. A format conversion of already-existing content is not expansion. That report was corrected, in the same terms as the incident above: no new content, no integration, no finished package, therefore *failure*, not "partial progress."

The mechanism that caught it was not a validator. It was a human refusing to accept the report. That is the limit of what mechanical verification can do on its own: Bridle can prove a file exists or does not.

It cannot yet judge whether the file is the *right* file for what was asked — that gap is a missing layer, not a solved problem, and this book should not claim otherwise.

The corrected rule

Before any chain, human, or agent reports a task as done, three questions, in order:

```
1. Was the agent even pointed at the right object?
   (Not: did it do something reasonable with whatever it saw.)

2. Does a mechanical check confirm the required artifact exists,
   in the required state, with no forbidden substitute present?

3. Having opened the artifact yourself — is it actually the thing you asked for?
```

Skipping question 1 produces a chain that works hard on the wrong problem. Skipping question 2 produces a report that trusts narration. Skipping question 3 produces a validator that is satisfied by something technically present and substantively wrong. All three are required. None of them is optional, and none of them is sufficient alone.

Part XVI — MetaVibing MetaVibes Itself

This part documents the production of this repository as a real case study of the method the rest of this book describes — not proof of some separate framework, but the plainest example there is of MetaVibing's own core loop applied to itself.

What Actually Happened

Every one of these is a real correction, in the order it happened, each one following the same loop from page one of this book: a mistake, a correction, and — where the correction was worth keeping — a durable artifact instead of a repeated conversation.

The README claimed files that didn't exist. Early drafts described `experiments/`, `patterns/`, and `templates/` directories, and a packaged manual edition, that hadn't actually been built. An external review caught it. The fix wasn't a better sentence — it was a rule for this project going forward: describe what's here, not the destination, and mark what's planned as planned.

The architecture checker silently missed real violations. Its documented invocation resolved the wrong directory for its own missing-test check, so an entire category of violation reported zero results no matter what was actually in the code. Nobody asked the model whether the checker worked — a real unit test, run against real code, found the exact bug (see Part III's "Add One Missing Tool," and `tools/architecture-checker/test_checker.py`).

The `.claude/` stack existed in the wrong layout. Rules, Skills, and an Agent were written, described as "live" in this very manual, and could not actually be loaded by Claude Code — wrong directory, no frontmatter. Prose about the stack was mistaken for the stack. Fixed by moving each artifact to the path and frontmatter Claude Code actually requires.

A frontmatter key was wrong, and only a real session caught it. Even after the layout fix above, the TaskFlow Rule used a `globs:` key that Claude Code doesn't recognize — it silently fell back to loading the Rule unconditionally instead of scoping it to `examples/taskflow/`. This was not caught by re-reading the file, or by asking a model to check its own work. It was caught by an actual fresh Claude Code session running the actual `/meta`, `/ship-change`, and TaskFlow-editing behaviors and reporting, honestly, that the fourth one didn't match its documentation. See `FRICTION_LEDGER.md`, F-003.

The Friction Ledger went from a template to a real record. For weeks it said "no entries yet," which was itself a small instance of the same failure this book warns about — an artifact that looks finished but holds no real content. It now holds real entries, including all four above, each one tagged corrected, mechanically verified, or behaviorally evaluated rather than a single undifferentiated "done."

None of these corrections required inventing a new mechanism. Each one is a plain instance of a pattern already named earlier in this book — a Rule, a test, a directory fix, a behavioral check, a ledger entry — applied to the project that was, at the time, writing about applying them.

What Has Not Been Demonstrated

This alpha will not claim more than it has shown. As of this edition:

The 3×3×2 evaluation protocol has not been run. No Condition A vs. Condition B comparison data exists. The claim that MetaVibing reduces architectural violation rate and correction turns has not been empirically tested. It is a hypothesis, not a finding.

This manual has not been exhaustively copyedited. It has been revised repeatedly against real evidence — see above — which is a different thing from a single, polished editorial pass.

No second practitioner has reproduced any eval result. The reproducibility standard specified in `evals/baseline/README.md` requires that a second experimenter be able to reproduce grading judgments from saved artifacts alone. No such reproduction has been attempted, because no trial has been run yet to reproduce.

The Evidence Hierarchy

MetaVibing applies a strict hierarchy to what counts as evidence, used throughout this part and the rest of the book:

```
Agent narration      →  testimony (weakest)
Validator result     →  evidence (mechanical)
Human judgment       →  authority (strongest)
```

When you read a report that says "completed" — ask what class of evidence supports it. If the answer is agent narration only, you have testimony, not evidence. If a real test ran, or a real fresh session behaviorally confirmed it, you have something stronger. Every correction listed above only counts because it cleared that bar — a test, a rendered page, a real session's report — not because an agent said so.

The Remaining Gap

There is a gap between what Bridle can validate mechanically and what humans must judge. Bridle can confirm:

- Required files exist at the required paths.
- Forbidden paths were not touched.
- Word count thresholds were met.
- Required sections are present by string match.

Bridle cannot confirm:

- Whether the content is accurate or useful.
- Whether the examples are meaningful or illustrative.
- Whether the evaluation protocol is well-designed.
- Whether the core claim is actually true.

That gap is not a defect to be engineered away. It is the permanent role of the human at the governance layer. The goal of MetaVibing is not to eliminate human judgment. It is to ensure that human judgment is applied to the right things — the things that actually require it — rather than being spent on mechanical verification that a validator can do faster and more reliably.

This is the deepest version of the central discipline from Part XIV:

> *What part of the system allowed this class of work to require human attention?*

Sometimes the answer is: it requires human attention because it genuinely requires human judgment. No artifact resolves that. The artifact only ensures the human is given real evidence to judge, rather than agent testimony to accept or reject on faith.

Conclusion — Code Builds the Product; Meta-Code Builds the Coder

The first phase of generative programming was about astonishing output.

Ask.

Generate.

Edit.

Ship.

The next phase is about accumulated intelligence.

Every serious interaction with an agent creates information about:

```
what it should know
how it should work
what it should never do
which tools it needs
which roles should exist
how its results should be judged
```

If that information disappears when the conversation ends, we are wasting it.

MetaVibing captures that information.

It converts experience into structure.

```
Mistake
   ↓
Rule

Repetition
   ↓
Skill

Expertise
```

```
   ↓
Agent

Invariant
   ↓
Hook

Blindness
   ↓
Tool

Pattern
   ↓
Plugin

Uncertainty
   ↓
Evaluation

Accumulated friction
   ↓
MetaAgent
```

The deepest shift is therefore not:

> *AI writes more code.*

It is:

> *AI participates in improving the machinery by which AI performs work.*

Humans move progressively upward in the stack.

From typing syntax.

To describing intent.

To designing workflows.

To governing agents.

To governing systems of agents that can themselves propose how those systems should evolve.

That is why Vibe Coding is not the endpoint.

It is time for MetaVibing.

And we are entering the MetaAgents Era.

Appendix A — The Master MetaVibing Prompt

```
You are now operating as a META-ENGINEER.

Your task is not primarily to improve the application.
```

```
Your task is to improve the Claude Code environment through which the
application is developed.

Inspect the available evidence:

- recurring user corrections;
- CLAUDE.md;
- rules;
- auto-memory;
- Skills;
- subagents;
- hooks;
- permissions;
- MCP;
- plugins;
- tests;
- project scripts;
- previous failures.

For each structural weakness ask:

1. Is this genuinely recurring?
2. What mechanism is missing?
3. What is the smallest artifact that would solve the class of problem?
4. Does an existing artifact already solve it?
5. Would this change create conflicting instructions?
6. Should this be soft guidance or deterministic enforcement?
7. Can the improvement be evaluated?
8. Can it be reverted?

Use this mapping:

persistent fact → CLAUDE.md
contextual fact → rule
procedure → Skill
specialist role → subagent
hard invariant → hook / permission
missing capability → MCP
reusable package → plugin
uncertain improvement → evaluation

Produce:

OBSERVATION
EVIDENCE
ROOT CAUSE
PROPOSED META-CHANGE
WHY THIS MECHANISM
ALTERNATIVES REJECTED
RISK
EVALUATION PLAN
ROLLBACK PLAN

Do not implement until explicitly authorized.
```

Appendix B — MetaVibing Review Prompt

```
Review this proposed Claude infrastructure change as if it were
production software.

Check:

- Does it solve an observed recurring problem?
- Is the mechanism appropriate?
- Is there a smaller solution?
- Does it duplicate existing instructions?
- Does it conflict with another rule?
- Does it unnecessarily consume context?
- Does it broaden permissions?
- Could it create unexpected automatic behavior?
- Is its invocation criterion clear?
- Can we evaluate it?
- Can we revert it?
- Does it preserve human authority?

Return:

APPROVE
APPROVE WITH CHANGES
REJECT

Then explain why.
```

Appendix C — The MetaVibing Golden Rules

Never suffer the same Claude failure three times.

Turn recurring corrections into artifacts.

Facts belong in memory; procedures belong in Skills.

Roles become agents.

Hard invariants become permissions or hooks.

Repeated copy/paste becomes a tool connection.

Reusable meta-infrastructure becomes a plugin.

Never confuse more infrastructure with better infrastructure.

Meta-code must be evaluated like product code.

Claude may propose its own evolution; it should not automatically grant itself authority.

Every important meta-change needs a rollback path.

Keep the human at the governance layer.

Improve the environment, not just the answer.

Code builds the product. Meta-code builds the coder.

Appendix D — Evaluation Status Summary

This appendix records the honest state of what has and has not been validated. Updated as of this alpha; see `FRICTION_LEDGER.md` for the live, dated version of this table.

Confirmed

Item	Status	Evidence
Companion repo tests (8 tests) pass	■ Confirmed	`examples/taskflow/` — 8/8 passing on the alpha release commit
Evaluation charter is complete and specific	■ Confirmed	`evals/baseline/README.md` — core claim, tasks, metrics, protocol, pass/fail gates
Meta-stack artifacts exist and are natively loadable	■ Confirmed	`.claude/rules/`, `.claude/skills/`, `.claude/agents/` present and behaviorally verified (FRICTION_LEDGER.md F-003)
Architecture checker CLI works	■ Confirmed	Returns 20 violations (17 db-in-handler + 3 missing-test) on unmodified TaskFlow, project-root invocation
Grader rubric (`evals/graders/rubric.md`)	■ Confirmed	7 atomic rule IDs, delta-based mechanical counting for 2 of 7

Pending

Item	Status	Blocker
Condition A baseline trials (T1, T3, T6)	■ Not run	First trial not yet executed
Condition B MetaVibing trials	■ Not run	Condition A must run first
M1 architectural violation rate	■ Not measured	Requires trial data

M2 correction turns per task	■ Not measured	Requires trial data
M3 pytest pass rate on first submission	■ Not measured	Requires trial data
Exhaustive manual copyediting	■ Pending	Ongoing
Architecture checker MCP server wrapper	■ v1.1 item	Not yet needed by any real friction
Second-experimenter reproducibility check	■ Not attempted	No second experimenter yet

This table is honest. The version of this manual that hid this table from you would be object substitution — reporting a more favorable state than the evidence supports.